BOQ Pipeline — Implementation Plan

Aetraa · Bills of Quantities from building drawings
Version 2.0 · 8 September 2026

1 · What we are building

A folder of drawings goes in. A Bill of Quantities comes out.

More precisely: a folder of split single-sheet PDFs, measured to POMI, producing a BOQ workbook — run by a single command, unsupervised, on any machine, at a cost that can be stated before the run begins.

It is built as a production system for people at Aetraa to use, not as a tool driven from a chat session. That distinction shapes every decision below.

Why we are building this rather than extending what exists

The delivered plugin carries a genuinely valuable body of material: the POMI clause library, a formula against each clause, the bill format measured off eleven issued tender workbooks, and a set of automated checks proving the clause library is faithful to the published standard.

None of that is in question, and we are keeping the substance of it.

What the plugin does not have is a way to run. Its measurement stage is not a program — it is a set of written instructions that a person follows inside a chat session, dispatching work across twenty-five nominal roles, with no fixed sequence, no record of progress, no spending limit, and no way to pick up where it left off after a failure.

The plugin's own audit puts it more sharply than we would. Tracing what actually calls what, it found that four of the steps in the chain are invoked by the written instructions rather than by code — **so the whole chain's enforcement rests on four prompt-controlled calls.** The one that matters most is the markup check, because it is the only step that independently re-derives a quantity, and therefore the only one that can catch a number that is simply wrong rather than merely undeclared.

That is the difference between a skill and a system, and it is why the two reported complaints — that the tool invents quantities, and that running it from a terminal does not work — are both consequences of a single cause.

The measurement *rules* and the *machinery that applies them* are separable things. The rules are the strength of what was delivered and we inherit them. The machinery is what we build.

2 · What we inherit, what we discard, what we do not have

Inherited

The clause library	314 POMI clauses with their text, taken as given
The formula against each clause	the unit it fixes, and the arithmetic that produces the quantity
The POMI section structure	the seventeen sections and their order in a bill
The house bill format	column widths, fonts, page structure and frozen wording, measured off issued workbooks
The traceability principle	every quantity names the drawing it came from. The right instinct, and we keep it
Four ideas from the determinism package	received 8 September: the separation of decision from arithmetic, an identity for the input set, a record of what the run actually did, and provenance as a deliverable rather than as bookkeeping. Sections 3, 4.1, 4.16 and 8

Discarded

The twenty-five seats	a description of how an office is organised, not something a machine can execute
Chat-session execution	the pipeline as instructions a person follows, rather than code that runs by itself
The duplicated script paths	the package contains two parallel sets of scripts. Only one has a way to start it, and the documentation describes the other. Its own handover asks which is canonical, because reproducibility can only be tested against one
The existing check suite	twelve automated checks, all verifying that the POMI library is faithful. Not one asserts that a folder of drawings produces a workbook. They answer a question we are not asking
The shared-constants register	it exists to stop one threshold being defined in two places with no link between them. That is a real problem in a codebase that grew; it is not a problem we will create

Not available to us

Stated plainly so no later reader assumes otherwise.

- **The RICS publication.** Copyrighted, and not supplied. We cannot independently verify the clause library against its source, and per instruction we accept the clause text as correct.
- **A reference BOQ.** We have not been given a bill considered correct. Until we have one we cannot say whether our output is right — only that it is internally consistent. Section 13.
- **A splitter.** Drawings arrive as combined multi-sheet PDFs and must be burst into one sheet per file before any of this runs. No code for this exists in anything delivered. The current dataset arrives already split. Building it is deferred.
- **CAD files.** Inputs are PDF. DXF or DWG would make a substantial part of this problem easier, and we do not have them. Section 4.8 sets out exactly what that costs.

3 · The core principle

The model identifies. Code computes.

Every stage falls on one side of that line, and the line does not move.

A model may say *this closed shape is a room*, or *this symbol is a floor drain*, or *this is a 200 mm hollow block partition, governed by clause D2.1*. Those are acts of recognition, made against evidence already pulled off the drawing.

A model may never say *this room is 24.5 m²*. It never produces a quantity, a dimension, a total or a deduction. Arithmetic is done in code, on values extracted mechanically, using the rule the governing clause fixes.

This principle has now been arrived at twice, independently. The determinism package delivered on 8 September reaches the same conclusion from the same evidence, and states it well:

> Stage 2 is Claude reading drawings, and that will vary between runs. The fix is not to constrain the model harder; it is to stop the model producing numbers at all.

Two designs converging on the same answer, from the same complaint, is reasonable confirmation that the diagnosis is right.

Three things follow from the rule.

It removes the failure you have been seeing. A model that gets a classification wrong produces a wrong label — visible, and checkable against the clause it cites. A model that gets a quantity wrong produces a plausible number that looks exactly like a measured one. The first is a defect somebody can catch. The second is invisible.

It makes the output repeatable. The same drawings must give the same quantities on any machine on any day. Where numbers come out of code, they do. Where they come out of a model, running twice gives two answers and no check applied to either means anything.

It makes the work testable. Because every quantity has a stated derivation — this evidence, this clause, this rule — it can be re-run and compared against what it produced last time. Where a number comes from a model there is nothing to compare, and no way to tell a genuine change from the model simply answering differently.

Where the line has to be drawn more carefully than it first appears

Stopping the model stating the quantity is not by itself enough, and this is the point on which our design and the delivered package differ.

If the model stops stating *the slab is 412 m³* but still states *the slab area is 137 m²* and *the thickness is 300 mm*, nothing has been fixed. The arithmetic is now honest and traceable, and the number is just as wrong — and it now carries a clause reference and a decision number, which make it look **more** credible than it did before.

So the rule extends: **the values a quantity is computed from must themselves be extracted, not asserted.** Section 4.9 sets out how, and what we do about the cases where extraction is genuinely not possible.

---

4 · The pipeline

Sixteen stages. Each takes a defined input, produces a defined output, and either runs mechanically or makes one bounded model call.

Throughout: a stage that cannot complete for a sheet records a **gap** — a positive statement that something could not be determined, naming the sheet and the reason. Gaps appear in the output. Nothing is silently absent and nothing is filled with an assumption.

4.1 Register

In: a folder path. **Out:** one record per file, and an identity for the set.

Walk the folder. Read each title block — drawing number, sheet title, revision, scale, discipline — and classify the sheet: drawing, cover sheet, transmittal, general notes, legend, authority certificate. Non-drawings leave the pipeline here but stay on the register.

Then resolve revisions. Where a drawing number appears more than once, the latest revision is measured and the others are recorded as superseded. This matters immediately: the current dataset has a preliminary submission folder sitting beside the approved one, and a run that measured both would double the entire architectural package.

The set is then given an identity — a single digest computed over every input file. This is one of the ideas worth taking from the determinism package, and its justification is the clearest sentence in it: *"same inputs, same outputs" is unfalsifiable if "same inputs" cannot be established.* Two runs can only be compared if they can first be shown to have measured the same drawings. It is twenty lines of code, and it turns every claim about repeatability from a statement into a test.

Mechanical, with a model as fallback. Title blocks sit in a conventional position and follow conventional patterns, so pattern matching resolves most of them. A sheet that does not parse goes to a single model call for that sheet alone, and the answer is stored so it is never paid for twice.

Project details are captured here — client, project, building, package, revision, date. Some of it reads off the title blocks. What does not is asked for once at the start of a run rather than guessed at.

If it fails: a sheet with an unreadable title block becomes a gap and is left out of measurement, rather than measured under an unknown identity.

How we prove it works: the drawing register you supplied lists every sheet in both issue sets with its number and title — 385 rows for the tender set. Our register must reproduce it. That is a real check, with a known right answer, on real data, available on day one, and it needs no model at all.

4.2 Probe

In: a registered sheet. **Out:** what can be read from it.

Every PDF falls into one of four classes, and which one governs everything downstream. This is a property of the file rather than a judgement about the drawing.

Native CAD	a DXF or DWG. Carries real, named geometry. Best case, and not present in our inputs
Vector PDF with text	real lines at real coordinates, and words that can be pulled off the page with their positions
Vector PDF, text outlined	real geometry, but the words have been converted into shapes and can no longer be read as words
Raster only	a picture of a drawing. No geometry to interrogate, no text to pull

If it fails: a raster-only sheet yields nothing measurable. That is the correct outcome, reported loudly, not something to work around.

4.3 Extract

In: a sheet and its class. **Out:** the raw evidence.

Pull every word with its position on the page, and every drawn line with its coordinates. Nothing is interpreted and nothing is measured — this answers only *what is physically on this sheet*. Every quantity produced later points back to it.

If it fails: a corrupt or password-protected PDF fails here, is named, and becomes a gap.

4.4 Calibrate scale

In: a sheet's text and geometry. **Out:** the conversion from page units to metres.

Getting this wrong scales every quantity on the sheet by the same wrong factor, so it is established twice and cross-checked: the scale stated in the title block, and a printed dimension measured against the geometry it annotates.

If the two disagree, the stage refuses. A sheet with no verified scale yields no lengths and no areas at all, and that fact is carried forward — section 4.9 explains why it has to be.

If it fails: refusal, recorded as a gap naming the sheet. Never a guess.

4.5 Route

In: a sheet's title and content. **Out:** what the sheet is.

Floor plan, section, elevation, detail, door schedule, column schedule, area calculation, legend. This decides which reader runs on the sheet, and it is the first point where anything resembling judgement enters.

Split. Sheet titles are largely conventional, so a sheet titled `SCHEDULE OF DOORS-01` needs no model. Where the title is ambiguous, one small model call settles it, choosing from a fixed list of sheet types.

If it fails: an unroutable sheet is reported as unclassified rather than pushed through a reader that will misread it.

4.6 Read schedules

In: a schedule sheet's words and their positions. **Out:** structured rows.

Schedules state quantities outright, and they are the most reliable source in the drawing set. A large share of any real bill comes from them, and none of it requires scaling anything.

Rebuilding the table is mechanical: group the extracted words into rows by their height on the page, order each row left to right, and the grid re-emerges from the coordinates.

Split, in a way that is worth noting. Which column holds the quantity, which holds the mark, which holds the dimensions — on an unfamiliar layout that is a judgement, and it is made **once per schedule sheet**. Code then reads every row using that mapping. Thirty schedule sheets means thirty model calls, not thirty thousand rows read by a model.

If it fails: a schedule whose columns cannot be mapped is recorded whole as a gap, rather than half-read.

4.7 Count symbols

In: a plan sheet's geometry and its legend. **Out:** counts by type.

Sprinkler heads, sockets, light fittings, floor drains, sanitary fixtures. Finding repeated identical shapes is mechanical. Knowing what a shape *is* means reading the legend, which is one model call per sheet.

If it fails: an unlegended symbol is counted and reported as an unidentified repeated element, rather than being given a plausible identity.

4.8 Measure from the drawing — and its limits

In: a plan sheet's geometry and its calibrated scale. **Out:** areas and lengths.

Wall runs, floor and slab areas, pipe and duct runs, taken off the drawn geometry. This is the hardest stage in the pipeline and the plan does not pretend otherwise.

The problem is that a PDF has no names. In a CAD file, every line sits on a named layer — a line on `A-WALL-INT-200` announces that it is a 200 mm internal wall. When the drawing is printed to PDF, those names are gone. What survives is coloured lines, and nothing on the page distinguishes a wall from a grid line, a dimension leader, a hatch boundary or the edge of the title block. Recovering the meaning is inference, and inference is where wrong numbers come from.

How we handle it. Lean hard on the two stages above. Schedules and counts are reliable, need no scaling, and cover more of a real bill than is generally assumed. Where measurement off the drawing is genuinely required, the value produced is marked as **inferred** rather than extracted, and that marking

travels with it all the way into the deliverable.

This stage will need iteration, and that is planned work rather than a caveat. It needs a set of sheets with hand-verified correct answers, tuning measured against that set, and honest reporting of where it fails. It is the part of this plan most likely to change shape once it meets real drawings.

What would remove the problem entirely: DXF files. If the native CAD can be obtained for even part of a set, most of this stage stops being inference and every quantity it produces becomes exact rather than estimated. It is worth asking for, and worth asking early. We are building for PDF because PDF is what we have.

If it fails: geometry that cannot be interpreted confidently produces no value and a recorded gap, never a plausible one.

4.9 The evidence base

In: everything the three readers produced. **Out:** the addressable set of values any quantity is allowed to be computed from.

This stage is what makes the core principle actually hold, and it is where our design departs most from the delivered package.

Every value that could feed a quantity becomes a numbered record carrying the value, its unit, the sheet it came from, where on that sheet, and — the important field — **how it was obtained**:

Stated	printed on the drawing and read directly. A schedule row saying 12 nr, a dimension string, a thickness on a section
Counted	produced by counting repeated symbols, with the legend read to name them
Inferred	measured off geometry whose meaning had to be reconstructed. Section 4.8

A quantity may only cite records from this set. It cannot be computed from a number that does not appear here, which makes an untraceable quantity structurally impossible rather than something a check hunts for afterwards.

Why this matters more than it looks. The delivered package has the model write the evidence base. That stops the model stating the quantity, but it still has the model stating the numbers the quantity is computed from — so a misread slab area of 137 m² simply becomes a misread evidence record of 137 m², now wearing a decision number and a clause reference that make it look more credible than before. The problem has moved one step back rather than gone.

Ours is machine-produced wherever it can be. A schedule row is read from the table. A symbol count is counted. Neither is a number the model chose. Only the inferred class involves reconstruction, and it is labelled as such, so a reviewer can see exactly which quantities in a bill rest on it.

Two rules enforced here.

A record from a sheet whose scale never calibrated **cannot be cited for a length or an area**. The delivered package identifies this well: such a figure is *a guess with a reference*. In our design it is prevented at construction rather than caught later.

A record's origin is never upgraded. An inferred value stays inferred through classification, computation, assembly and into the provenance sheet in the delivered workbook.

How we prove it works: every record resolves to a real sheet and position; no record cites a blocked sheet for a length or area; and the counts of stated, counted and inferred records are reported at the end of every run, so the composition of a bill is visible rather than assumed.

4.10 Build the inventory

In: the evidence base. **Out:** one consolidated list of real elements.

Each element carries its mark, type, dimensions, count, source sheets and evidence records. This is where the pipeline stops thinking in sheets and starts thinking in building.

Three things happen here. The second and third are where bills go badly wrong.

Reconciliation across sources. The door schedule says 47, the plan count says 44. That surfaces as a reported discrepancy — never averaged, never silently resolved in favour of whichever source ran last.

Floor multipliers. A sheet titled *TYPICAL FLOOR PLAN (2ND, 4TH & 6TH)* is one drawing standing for three floors. Booked once, the building loses two thirds of everything on that plate. The current dataset contains exactly that sheet. So the inventory carries an explicit multiplier per plan, read from the sheet title, and **a plan whose floors cannot be determined is a gap, never assumed to be one.** The delivered notes put the undercount from getting this wrong at between 16% and 58% on the reference project, which is the difference between a usable bill and a worthless one.

Cross-sheet double counting. The same wall appears on a plan, in a section and again on a detail. Counted three times it is billed three times. Elements are matched across sheets and merged, and a count that exactly equals another element's count is flagged for inspection before it is measured, because that pattern usually means one thing has been read as two.

Mechanical throughout — all three are rules applied to data already extracted.

If it fails: an unresolvable discrepancy is reported as a discrepancy. The pipeline does not pick a winner.

4.11 Classify to POMI

In: an element and its evidence. **Out:** the governing clause, and which records feed it.

This is the last point at which a model touches anything that becomes a number, and its role here is deliberately narrow.

The model supplies two things and nothing else: which clause governs this element, and which evidence records are its inputs — this record is the area, that one is the thickness.

Everything else comes from the clause. The unit, the arithmetic, any threshold that decides whether the item is measured at all, the deduction rule, and how items are grouped into separate bill lines are all properties of the clause POMI fixes. They are looked up, never chosen.

This is a deliberate simplification of the delivered design, and the register supports it. That design has the model choose the clause *and* the operation, then checks the operation against a list of about ten permitted forms. Reading the formula register shows why the choice is unnecessary: of 314 clauses, 210 carry a formula, and **only 42 involve arithmetic beyond adding up measured values.** The remaining 168 are variants of *sum the lengths* or *count them*. And in every one of the 42, the arithmetic is fixed by the clause — C2.1 is always volume as number times length times width times thickness. A model asked to choose it can only agree or be wrong.

Removing the choice removes a whole category of error: the model can no longer pair a correct clause with an incorrect operation.

Classification calls are batched — one call covering many elements of the same family, not one per element.

If it fails: an element that cannot be confidently classified is carried into the output as unclassified with its evidence attached, so it can be resolved by a surveyor, rather than being assigned a clause that may be wrong.

How we prove it works: every clause the model returns must exist in the clause table; the evidence roles it supplies must be the roles that clause's formula actually requires, in the right number; and every record it cites must exist. A model that invents a clause, or supplies a thickness to a clause that measures by length, fails immediately and visibly.

4.12 Compute

In: classified elements and their evidence records. **Out:** quantities.

Apply the clause. That means four things, and only the first is arithmetic.

The formula. For 168 clauses this is summing the measured values. For the other 42 it is a specific expression the clause fixes — area by thickness, number by length by width by depth, weight from a length and a per-metre mass, the circular ones by diameter.

The threshold tests. Several clauses make an item's existence conditional on a measurement. A groove below a stated sectional area is deemed included and must not be billed. Pipe fittings below a stated internal diameter are deemed included; above it they are counted and grouped by size. **These are where the money is** — and the delivered package's own changelog records exactly such a clause being documented backwards, in a way that would have billed a class of work POMI gives away. They are implemented in code, with test cases sitting either side of every threshold.

The deductions. What comes off a gross area is a POMI rule, not a choice — voids at or above a stated area come off, and all voids at an edge come off at any size. The model identifies which openings exist; the standard decides which of them are deducted.

The grouping. Whether a clause produces one bill line or one per size, per class, per type. That determines the shape of the bill and it too is fixed by the clause.

Rounding happens exactly once, at the end, at the precision POMI fixes. Nothing rounds intermediately. Two roundings applied to the same chain diverge in the last decimal, and that is one of the standard causes of two runs disagreeing.

Entirely in code. No model, at any point, for any reason. This is the stage the core principle exists to protect.

If it fails: an element missing a value the clause requires produces no quantity and a gap naming what is missing. It does not fall back to a default.

How we prove it works: this gets the largest and most detailed set of automated checks in the project. Every one of the 42 arithmetic clauses gets cases, every threshold gets cases on both sides of it, and every deduction rule gets cases. The worked examples in the clause document are themselves a ready-made set of test cases — with the caveat that at least one has already been found to state the opposite of what its clause produces, which is an argument for testing against the clause text rather than against the example.

4.13 Write descriptions

In: a classified element and its attributes. **Out:** the bill line's wording.

Contractors price off this wording, so it carries real money, and you have already identified descriptions reading unlike a surveyor as a defect in the current output.

Model, tightly constrained. Not free writing: a template per clause family with slots, and the model fills the slots from the element's own attributes. Every filled slot must trace back to an attribute the element actually carries. A description containing anything unevidenced is rejected before it reaches the bill.

If it fails: a slot that cannot be filled leaves the description incomplete and flagged, rather than plausibly completed.

How we prove it works: automatically, that no slot contains content the element does not carry. The quality of the wording itself is reviewed by a person — we say so rather than pretending a script can judge it.

4.14 Items that come from no drawing

In: the project details. **Out:** the unmeasured sections of the bill.

Preliminaries, dayworks and contingencies are not measured off a drawing. They are stated items and sums, and POMI has sections governing all of them.

A pipeline that only reads drawings ships a bill missing its opening section entirely. This stage exists so that does not happen: the required items are generated from the POMI structure and the project details, carried as stated items, and left blank and named where the value is a commercial decision rather than a measurement.

Largely mechanical — it is structure driven by the standard, not inference from evidence.

4.15 Assemble

In: every bill line. **Out:** the bill's structure.

Group by section, order the sections as POMI orders them, letter the items, paginate, and build the collection carrying section totals to the summary. The lettering convention skips certain letters and restarts on each page; these are house rules and they are applied exactly.

Mechanical.

If it fails: a section total that does not match its own items halts the run rather than producing a bill that does not add up.

4.16 Render

In: the assembled bill. **Out:** the workbook.

Deliberately thin, and deliberately last. The output must match the format of a BOQ you consider correct, and we have not yet been given one. It is built as a formatting layer over a stable internal structure, so that when the reference bill arrives the work is formatting rather than rebuilding.

One addition beyond the bill itself: a provenance sheet. For every billed line — the clause applied, the evidence records it was computed from, the sheets those came from, and whether each was stated, counted or inferred. It is built from the working records rather than read back out of the workbook, so it cannot inherit an error the workbook already carries.

This is worth building for a specific reason. Every Aetraa output is signed off by a senior professional before it leaves. A figure queried in a tender review can then be answered by one lookup instead of a reconstruction, and the reviewer can see at a glance which quantities rest on a schedule and which rest on a reading of unnamed geometry. Those are not equally defensible, and the person signing is entitled to see which is which.

---

5 · How this was built, and how we are building it

The comparison matters because the existing design is coherent. It is not a poor attempt at what we are doing — it is a good attempt at something else. It optimises for the *defensibility of the measurement rules*. We are optimising for *the run finishing, repeatably, at a known cost*. Nearly every difference below follows from that one divergence.

	As delivered	As we are building it	What it means
How it runs	Written instructions followed inside a chat session	A program with a start and a finish	It can be run by someone who is not troubleshooting it
What enforces the chain	Four of the steps are invoked by the prompt, not by code	Every step is called by the program	A step cannot be quietly skipped
Unit of work	Batches of fourteen sheets through an office of twenty-five roles	One sheet through a fixed sequence of stages	Progress is countable, and a failure points at a sheet and a stage
Where numbers come from	A model, reading drawings and reporting quantities	Code, applying the clause's own rule to extracted values	A wrong number becomes repeatable and locatable instead of a one-off
Where the inputs to a number come from	Also the model	Extracted from schedules and counts where possible, and labelled where not	The problem is removed rather than moved one step back
Who chooses the arithmetic	The model picks an operation from a permitted list	Nobody. The clause fixes it	A right clause can no longer be paired with a wrong operation
Reference material	Thousands of lines loaded in full, repeatedly	A structured table for lookups; the relevant section's guidance only where a judgement is being made	Cost per call drops sharply, and stops growing as the documents grow
Memory of progress	None. A run is a conversation	Each stage's output written to disk, with a record of what ran	An interruption costs one sheet, and a skipped stage is visible
When something fails	The run stops, or the model works around it	Named, retried if temporary, recorded as a gap if not	A partial bill with a stated list of holes, rather than nothing
Cost	No limit, no estimate, no measurement	A known ceiling, estimated before and measured after	The cost of a run can be stated before it is run
Dependence on the model	Total. The pipeline *is* the model	Bounded, at specific classification points, behind one interface	Changing model or supplier is a settings change
What is verified	That the POMI library is faithful — twelve checks, none about running	Both: that the rules are applied correctly, and that drawings produce a workbook	"Does it work" becomes a question with an answer
Can it be tested	Barely. A quantity from a model has nothing to compare against	Every quantity has a derivation that can be re-run and compared	A real regression is distinguishable from the model just answering differently

The last row is the one that compounds. Everything above it follows from where the numbers come from.

6 · The POMI data

The source

The 151-page PDF and, inside the plugin, the markdown file it was generated from. Same document; the markdown is the newer of the two and keeps its structure intact where the PDF has reflowed text and damaged characters. **We work from the markdown.**

It holds, against each of 314 clauses: the clause text, the rule, the formula, the take-off wording, the markup convention and the bill wording. Alongside POMI's own text sit the house decisions — where the standard offers a choice and the house has made one, or where the standard is silent and the house has extended it.

Per instruction we take this document as given: the clause text is verbatim from the RICS publication, and the formulas are correct. We are not positioned to verify the first, and we are not reopening the second.

The ingest

A **one-off** read of that document, run once, its output saved into the project, repeated only when a new version of the document arrives. It is not part of a BOQ run.

It produces two things, because the document holds two kinds of material wanting opposite treatment.

A structured clause table — clause, unit, formula, threshold tests, deduction rule, grouping rule, section. Read on every item during classification and computation. It has to be exact and searchable, which a 7,000-line document is not. Section 4.12 depends entirely on it.

Guidance held per section — the traps, the worked examples, how each section is taken off. Given to a model at the moment it makes a judgement in that section, and only that section. A judgement about section C receives section C's guidance, never the whole document.

What the register actually contains

Worth stating, because it sizes the work in section 4.12 and it is not what a clause count suggests:

314	clauses in total
210	carry a formula
42	involve arithmetic beyond summing measured values
168	are variants of *sum the lengths*, *sum the areas*, *count them*
104	carry no arithmetic at all, and say why

So the arithmetic to build and test is 42 specific clause rules plus one default. The difficulty is not the volume of formulas — it is the threshold tests and deduction rules that sit alongside them, and those are where a bill loses money.

Values that could change, and how we treat them

The history of this material shows it moving. Flagged here so nothing is mistaken for a permanent fact.

Value	Status	How we handle it
314 clauses	The current count, and the document is generated to match it	Taken from the ingest, never written into the code. A new version with a different count gives us a different count
How many clauses carry a formula	Stated as 210 in one place and 207 in another, within the same delivery. Reading the register gives 210	Counted from the data. We do not assert a number we did not derive
Section letters	**Already changed once.** Version 3.0.0 corrected several section letters and dissolved one section, against bills already issued under the old lettering	Held as data. Any comparison against an older bill needs that mapping applied, and we note it where it arises
Bill format constants	**Already found wrong once, by the client.** A re-measure against the issued workbooks showed a recorded column width was the third most common value, not the first	We take the current values and treat them as a belief to confirm against the reference BOQ when it arrives
Deduction thresholds	Specific numbers, in a document one adversarial review has already found eight errors in	Taken from the ingest, with automated checks sitting either side of each threshold, so a changed number produces a visible failure rather than a quietly different bill
The worked examples	At least one has been found to state the opposite of what its clause produces	Used as test cases, but checked against the clause text rather than trusted outright

The pattern: **derive, never hardcode.** Where a number has to be relied on, it comes from the ingested data and has a check on it, so a future version of the document breaks something visibly instead of changing a bill silently.

---

7 · The model layer

One door

Every model call goes through a single module. Nothing supplier-specific reaches any stage of the pipeline, and no stage knows which model answered it.

Each call states what it expects back — a sheet type from a fixed list, a column mapping, a clause reference with its evidence roles, a filled description — and the answer is checked against that shape before it is used. An answer that does not fit is a failed call, retried or recorded, not a value passed downstream.

That is all being model-agnostic requires in practice. We run on Claude now because it is what is available and it is good at these judgements. Changing later is a settings change plus a re-run of the checks, because exactly one place knows how to talk to a supplier.

One call per judgement

Each judgement is made once. A response that fails its contract — an invented clause, a sheet type outside the list — is retried, because that is a malformed answer rather than a wrong one. A response that fits the contract is used.

We do not make the same judgement several times and compare. The delivered package proposes exactly that: run the measurement stage repeatedly and take agreement across passes as evidence. We are not doing it, for two reasons. It multiplies the expensive part of every run, permanently, to buy confidence that belongs in development rather than in production. And it measures the wrong thing — several passes agreeing shows the model is *consistent*, not that it is *correct*, and a systematic misreading agrees with itself perfectly.

The confidence comes from section 9 instead: measure accuracy against a set of known right answers during development, correct the prompt, and repeat until one call is reliable. That work is done once and costs nothing per run.

Repeated passes remain useful as a **development measurement** — running a sample several times shows which judgements are unstable, which tells us where to spend effort. That is diagnostic work done a handful of times, not a production feature.

Keeping calls small

Every call carries the minimum: the evidence for the judgement being made, and the relevant section's guidance. Never the full clause document, never the whole drawing set, never an accumulated conversation. This is the single largest cost difference from the current design, which loads thousands of lines of reference material as context and does so repeatedly.

Estimated number of calls

For the 385-sheet tender set. An order-of-magnitude estimate to be measured on the first real run, not a figure to be relied on in advance.

Stage	Calls	Size
Title block fallback	~20	small
Sheet routing	up to ~385	small
Schedule column mapping	~30	small
Legend and symbol naming	~150	medium
Reading geometry	~150	large
Element classification, batched	~40	medium
Descriptions, batched	~30	medium
Total	**~800**	~150 of them expensive

The number that actually matters is the **ceiling**. This is capped by the sheet count and the element count, both known before a run starts, so a cost can be estimated and displayed beforehand and a limit can be set. The current design has no ceiling — its cost depends on how the session happens to go.

8 · Running it

Doing sheets at the same time

Sheets are independent from registration through to geometry, so they are processed several at once rather than one after another. Consolidation onward is sequential, because it works on the building rather than on sheets. How many run at once is configurable, since the real limit is the model supplier's rate limits rather than the machine.

When things fail

An error handler that catches everything and writes "failed" is worse than none, because it turns a bug into a quietly missing quantity. So:

- Every failure names the **sheet**, the **stage**, and **what it was doing**.
- Failures are sorted into two kinds. **Temporary** — a rate limit, a timeout, a truncated answer — is tried again after a pause. **Structural** — a corrupt PDF, an unreadable title block, an unverifiable scale — is not, because trying again gives the same result. It records a gap.
- Nothing is caught in general. A handler exists for a cause somebody has thought about.
- An unexpected error is not swallowed. It fails its sheet loudly, is written to the log in full, and the run carries on with that sheet marked failed.

A run ends with a bill and a list of gaps, or with a stated reason it could not produce one. It does not end with a bill that quietly leaves out what it could not read.

What the run records about itself

Each stage writes its output to the run folder as it finishes, together with a record of what ran: which stages were entered, when, whether they succeeded, and what they produced.

Two things follow. A run can be resumed from the first incomplete stage, so an interruption at sheet 300 costs one sheet. And **a stage that did not run becomes a visible absence rather than an invisible one** — which closes the failure mode where a check never fires because it was never reached, and the bill is produced anyway with nothing in it saying so.

Ctrl-C stops cleanly.

The log

A record written to a file in the run folder, detailed enough to diagnose a failure two days later. What appears on screen is for the person watching; the log is the record.

Two habits enforced throughout

Never iterate an unsorted collection. Unsorted iteration is the most common cause of the same input producing different output, and it is silent.

Round exactly once, at the end. Two roundings applied to one chain diverge in the last place.

Both come from the determinism package, and both are the kind of thing learned by being bitten.

9 · How we prove new work does not break old work

Five layers.

Checks on every mechanical stage. Registration, extraction, scale, schedule reading, the evidence base, consolidation, computation, assembly. These carry the bulk of the work, and computation gets the most detailed coverage in the project — the 42 arithmetic clauses, every threshold tested on both sides, every deduction rule.

Checks on the clause data. The ingested table is compared against saved expected values. A changed formula, unit or threshold in a future version of the document becomes a visible failure naming the clause, rather than a quietly different bill.

A set of known right answers for the model. Built during development: several hundred real judgements taken from these drawings with the correct answer recorded — this sheet is a door schedule, this column holds the quantity, this element is governed by this clause. The model is measured against it, the prompt corrected, and the measurement repeated. This is where confidence in a single call is bought, and it is the reason section 7 does not need repeated passes at run time. It doubles as a regression check: a change to a prompt or to the model that lowers the score is visible before it reaches a bill.

Checks on the rules of the exchange. Separately from accuracy, a model answer must be structurally valid — a sheet type from the allowed list, a clause that exists in the table, evidence roles matching what that clause requires, a description containing nothing the element does not carry. We do not assert the model made the right call; we make it impossible for it to make an invalid one without failing visibly.

Checks on the whole run. The drawing register gives a real expected answer for registration across 385 real sheets, available immediately. Beyond that, a fixed set of sheets must keep producing the same output, re-run on every change — and because the mechanical half is identical by construction, any difference in that comparison locates itself at the model boundary.

---

10 · Code standards

Internal discipline, recorded so it is not negotiated later.

- **Python throughout.** One package. The terminal runner is a thin shell over it, so the later version uses the same code rather than a second copy of it.
- **An automated code-quality score of 8.5 or above on every file**, enforced rather than aspired to.
- **300 lines per file maximum.** A file approaching the limit is split before it becomes a problem, not after.
- **Documentation on every component** — what it does, what it takes, what it gives back, and anything a caller would be surprised by. Brief and accurate rather than narrated.
- **Proper logging throughout**, never messages printed ad hoc.
- **Dependencies pinned**, so that a library upgrade which changes how geometry is read shows up as a deliberate change rather than as quietly different quantities.

---

11 · The runner

Temporary, and built as such. It exists to exercise the pipeline during development and to let someone produce a bill before there is a proper interface. It will be thrown away, so it stays thin and holds no logic of its own.

```
1. Make BOQ
Q. Quit
```

Choosing **1** asks for a folder path. The path is checked — it exists, it is a folder, it has PDFs in it — and a bad path asks again rather than crashing back to the menu. Any project details that could not be read off the title blocks are asked for once here.

The run then proceeds, showing which stage is active and how many sheets are done, so it is visible that work is happening. Ctrl-C stops it cleanly.

On finishing it prints the path to the workbook, written beside the input folder, with a count of recorded gaps. Then:

```
1. Make another BOQ
Q. Quit
```

Developer options, not shown on the menu: run a single discipline or a stated number of sheets, since full runs cost real money and real time and this will be run hundreds of times during development; full detail on screen as well as in the log; and a mode that runs everything except the model calls, to exercise the mechanical path on its own.

---

12 · Build order

Each phase ends with something that runs. A bill-shaped output exists early and becomes more complete, rather than nothing working until everything works.

Phase	Delivers	Why here
1 · Ingest	The clause table — formulas, thresholds, deduction and grouping rules — and the section guidance, with checks	Everything downstream reads it. Nothing can be classified or computed without it
2 · Register, probe, extract, scale	A run over 385 real sheets producing evidence and a register, with an identity for the input set	Entirely mechanical, no model, and checkable immediately against the drawing register
3 · Runner skeleton	The menu, the path prompt, the progress display, the run folder, the run record, the log	Built now, because everything after it is developed by running it
4 · Schedules	Real values from real drawings, into the evidence base	The most reliable source, and the first point at which the pipeline measures anything
5 · Counts	Symbol counts by type	Same shape as schedules, extends coverage
6 · Classify and compute	Quantities against POMI clauses, with thresholds and deductions applied	The first genuinely bill-shaped output. Everything before this is evidence
7 · Consolidation	Multipliers, double-count removal, source reconciliation	Needs several sources present to mean anything, and it is where the costly errors live
8 · Measuring from drawings	Areas and lengths, marked inferred	Deliberately late. The hardest stage, needing iteration, and by this point there is a working pipeline to iterate inside
9 · Descriptions and assembly	A complete, structured, unpriced bill	
10 · Render	The workbook in the house format, plus the provenance sheet	Last, and dependent on the reference BOQ arriving

The set of known right answers described in section 9 is built alongside phases 4 to 8, as each model-dependent judgement is introduced. It is not a phase of its own, because it is only meaningful against a judgement that already exists.

---

13 · Open questions and known risks

We cannot yet say whether the output is correct. This is the significant one. We can prove the pipeline runs, that the arithmetic is right, that clauses resolve, that nothing is fabricated and that gaps are reported. We cannot prove the bill is a *good* bill until there is one considered correct to compare against. Until then the honest claim is internal consistency and traceability, not accuracy. **Obtaining a reference BOQ is the highest-value thing that could happen to this project.**

The output is unpriced, deliberately. A tender bill is issued with the rate column empty. Rates and pricing are out of scope here.

How much of a real bill comes from schedules versus measurement off the drawing is not known to us. It determines how much section 4.8 matters — if most quantities come from schedules, the hardest stage is a smaller share of the problem than it looks. The composition report described in section 4.9 answers this on the first complete run.

Splitting combined PDFs is neither built nor planned here. The current dataset arrives already split. A future one may not, and no code for it exists in anything delivered.

We develop against the tender drawings. The dataset also holds an approved set; the tender set is the cleaner of the two.